

BACKEND DEVELOPER

PRACTICAL EXAMINATION

Python | Django / FastAPI / Flask | REST API

EXAM CODE	BE-PY-01	MODE: ELECTRONIC	100 Marks
DURATION	3 Hours	FRAMEWORK	Candidate Choice
EXAM URL	https://acbt.com.ng/verify_access_tekhub		

1. Examination Brief

Challenge: Build a secure, database-backed REST API for a small service desk system. The API must allow authenticated users to create and manage support tickets, while an administrator can oversee all tickets and control workflow status. Your solution must be implemented in Python using Django, FastAPI or Flask.

Core objective

This is a backend engineering exam. Examiners are assessing API correctness, data modelling, security, validation, maintainability, testing and operational readiness. Framework choice will not affect scoring. Equivalent framework conventions are accepted when they satisfy the required behaviour.

Required domain model

- User: id, full_name, email, password hash, role (user or admin), created_at.
- Ticket: id, title, description, priority (low, medium, high), status (open, in_progress, resolved), owner, created_at, updated_at.
- Comment: id, ticket, author, body, created_at.

You may add fields that improve the design, but the required fields and behaviours above must remain available.

2. Rules and Restrictions

- Use Python and exactly one primary framework: Django, FastAPI or Flask. Django candidates may use Django REST Framework. FastAPI and Flask candidates may use standard ecosystem libraries for ORM, validation, authentication and migrations.
- No AI coding tools, AI assistants, code-generation extensions or AI-enabled IDE completion during the examination.
- Official Python, framework and library documentation may be consulted. Copying a complete project, tutorial solution or substantial third-party implementation is prohibited.
- No external assistance, pair programming, remote support or code sharing is permitted during the monitored session.
- Screen sharing must remain active and visible to the examiner throughout the practical session.
- Use a local database for the examination. SQLite is acceptable. PostgreSQL or MySQL may be used only if already configured and they do not consume examination time.
- Passwords must never be stored in plain text. Secrets, tokens and credentials must not be committed to the repository.
- The final project must start from the submitted repository using the documented setup steps without code repair by the examiner.

Automatic disqualification

Use of an AI coding tool, receiving external assistance, submitting copied work, deliberately concealing dependencies or falsifying the demonstrated solution may result in immediate termination of the examination.

3. Functional Requirements

A. Authentication and Identity

- Register a user with full name, unique email and password.
- Authenticate a registered user and issue a valid authentication credential. Token-based authentication is preferred; session authentication is acceptable only when correctly implemented for an API.
- Provide an authenticated endpoint that returns the current user profile.
- Reject invalid credentials and protected requests without valid authentication using an appropriate HTTP status code.
- Provide a documented method for creating or seeding one administrator account for examiner testing.

B. Ticket Management

- Authenticated users can create tickets with title, description and priority.
- Users can list and retrieve their own tickets. An administrator can list and retrieve all tickets.
- Users may update the content of their own tickets while the ticket is not resolved. An administrator may update any ticket.
- Only an administrator may change ticket status to in_progress or resolved.
- Deleting a ticket must be permission controlled. At minimum, an administrator can delete any ticket; a normal user must not delete another user's ticket.
- Ticket list output must support pagination and at least one useful filter such as status or priority.

C. Ticket Comments

- Authenticated users can add a non-empty comment to a ticket they are permitted to view.
- An administrator can comment on any ticket.
- Comments returned for a ticket must identify the author and creation time.

D. Service Health

- Provide a lightweight health endpoint, for example GET /health, that returns an HTTP 200 response and a simple service status payload.

4. API, Validation and Security Requirements

HTTP and API behaviour

- Use resource-oriented URLs and appropriate HTTP methods for create, read, update and delete operations.
- Return JSON responses with appropriate HTTP status codes. Do not return HTTP 200 for every outcome.
- Use consistent error response structures for validation, authentication, authorization and missing resources.
- Prevent ordinary users from accessing or modifying records they do not own unless the specification explicitly permits it.

Minimum validation rules

- Email must be valid and unique.
- Password must meet a reasonable minimum length and be securely hashed by the framework or an established password hashing library.
- Ticket title: 5 to 120 characters.

- Ticket description: minimum 20 characters.
- Priority and status must accept only documented values.
- Comment body must not be blank.

Security baseline

- Keep secret keys and configurable values outside source code where practical. Include a safe .env.example when environment variables are used.
- Use ORM or parameterized database operations. Do not construct SQL queries by directly concatenating untrusted input.
- Do not expose password hashes, secrets or internal stack traces in normal API responses.
- Apply authorization checks server-side. Hiding an endpoint in documentation or the client is not authorization.

5. Persistence, Architecture and Project Structure

Use the conventions of your selected framework. The following structure is a reference model, not a mandatory folder-for-folder requirement. Examiners will accept an equivalent professional structure.

```
backend-exam/  
  app/  
    config/  
    models/  
    schemas-or-serializers/  
    routes-or-views/  
    services/  
  migrations/  
  tests/  
  .env.example  
  requirements.txt or pyproject.toml  
  README.md
```

Database requirements

- Use persistent database storage. In-memory lists or dictionaries are not acceptable as the primary datastore.
- Define relationships and constraints so ownership and ticket comments are represented correctly.
- Provide migrations or the framework-equivalent reproducible schema setup.
- Use created_at and updated_at timestamps consistently where required.

Code quality expectations

- Separate configuration, data access, request handling and business rules where the framework makes this practical.
- Use clear naming, small focused functions or classes and avoid unnecessary duplication.
- Do not place the complete application in one oversized file unless the framework and scope genuinely justify it.
- Handle expected failures cleanly instead of relying on uncaught exceptions.

6. Automated Testing and Documentation

Minimum automated tests

Submit at least 8 meaningful automated tests. They must include both successful and failure cases. A strong submission should cover:

- user registration and duplicate email rejection;
- login with valid and invalid credentials;

- protected endpoint access without authentication;
- ticket creation validation;
- owner access versus another user's access;
- admin-only status change;
- comment creation or validation;
- one not-found or permission-denied case.

API documentation

- Provide interactive OpenAPI documentation when the framework supports it naturally, or provide a clearly documented endpoint reference in the README.
- Document authentication format, required request bodies, important response fields and how an examiner can create or obtain an admin account.
- If a Postman or Bruno collection is included, it is supplementary and does not replace a usable README.

Time management guidance

Establish the project and database first, implement authentication and core ticket CRUD next, then authorization, validation and comments. Reserve the final portion of the examination for tests, README verification, a clean Git commit and a fresh local run from the documented instructions.

7. Required README Content

- Candidate name and selected framework.
- Python version and package installation command.
- Environment configuration and database setup or migration command.
- Exact command used to start the API.
- Exact command used to run automated tests.
- Authentication method and endpoint summary.
- How to create or seed the administrator account.
- Any incomplete requirement or known limitation.

8. Submission Requirements

- Push the complete project to GitHub before the examination timer expires and submit the repository URL to the examiner.
- Repository naming format: firstname-lastname-backend-python-exam.
- Do not commit virtual environments, cache folders, local secret files, credentials, access tokens or unrelated files.
- Include requirements.txt or pyproject.toml with sufficient dependency information to reproduce the environment.
- The repository must be accessible to the examiner. If private, grant examiner access before the timer expires.
- The final commit used for assessment must be pushed before the timer expires and must match the code demonstrated during the monitored session.

9. Assessment Rubric

Assessment Area	Marks	What Examiners Check
-----------------	-------	----------------------



API functionality and completeness	25	Required authentication, ticket, comment and health behaviours work correctly across normal and failure cases.
Data modelling and persistence	15	Sound schema, relationships, constraints, migrations and reliable database persistence.
Authentication and authorization	15	Secure credential handling, protected endpoints, ownership checks and administrator permissions.
Validation and error handling	10	Input validation, consistent errors, appropriate HTTP status codes and no avoidable stack-trace leakage.
Architecture and code quality	10	Clear organization, naming, separation of concerns, maintainability and minimal duplication.
Automated testing	10	At least 8 meaningful tests covering success, validation, authentication, permissions and failure cases.
HTTP design and API documentation	5	Sensible routes and methods, useful API documentation and clear authentication usage.
Configuration and operational readiness	5	Reproducible setup, dependency management, environment configuration and reliable startup.
Submission quality	5	Clean repository, complete README, correct final commit and examiner can run the project without repair.

Suggested benchmark

70/100 indicates the minimum standard for progression. 85/100 or higher indicates strong production-ready backend performance for this practical.

10. Examiner Verification Checklist

- ☐ Project installs and starts from the submitted repository using the documented instructions.
- ☐ Candidate used Python with Django, FastAPI or Flask as declared.
- ☐ Registration, login and current-user endpoint work.
- ☐ Passwords are securely hashed and are never returned in API responses.
- ☐ Ticket create, list, retrieve and update behaviours work.
- ☐ Ordinary users cannot access or modify another user's protected ticket data.
- ☐ Administrator can access all tickets and perform the required privileged status operation.
- ☐ Ticket validation, missing-resource and permission errors return appropriate status codes.
- ☐ Comments work with the required access control.
- ☐ Pagination and at least one ticket filter work.

- ☐ Database persistence survives application restart.
- ☐ Migrations or equivalent schema setup are present and reproducible.
- ☐ At least 8 automated tests are present and executable.
- ☐ No secrets, credentials or local environment files are committed.
- ☐ No AI or external-assistance violation was observed during monitoring.
- ☐ Submitted GitHub repository matches the code demonstrated during the session.

11. Candidate Declaration

I confirm that the submitted work was completed by me during the monitored examination session and that I complied with the examination rules, including the prohibition on AI coding tools, copied project implementations and external assistance.

Candidate Signature

Date

Examiner Signature

